Arm A4: Mixture-of-Density

Marginal-mixture conditional flow matching in Flow-Factory

Jayce-Ping

2026-07-31

Abstract

Arm A4 is the deterministic probability-mixture alternative to transition-level P-OPD. Instead of assigning a likelihood to one ODE step, it mixes the *time marginals* generated by the old student and the teacher,

$$q_t = (1-\alpha)\rho_{\text{old},t} + \alpha\rho_{T,t},$$

and trains one student flow whose marginals follow q_t. The apparently difficult target velocity contains a density responsibility $\gamma_t(x) = \alpha\rho_{T,t}(x)/q_t(x)$, but conditional flow matching removes the need to estimate either density: sample one old/teacher branch for an entire trajectory and regress the student onto that branch's velocity.

This tutorial starts from the ODE and continuity equation, derives the exact two-source CFM identity, illustrates the construction with five diagrams, and distinguishes A4 from direct teacher regression, arithmetic field interpolation, SDE P-OPD, and per-step branch switching. The second half maps the mathematics to the implemented same-VAE XOPD path for FLUX.2-Klein: pre-rollout Bernoulli routing, one ODE trajectory per sample, rollout-time `noise_pred` capture, student-conditioning restoration, separate state/callback index maps, and a single student velocity-MSE pass. It also records the implementation's validation, evaluation, logging, and distributed call-order invariants.

The reader-facing presentation name for this construction is *Mixture-of-Density*. The tracked CPU-only companion is `mixture_of_density_demo.ipynb`.

Contents

1 The problem A4 solves

1.1 Learning goals

After reading this note, the reader should be able to answer five questions:

1. What distribution does A4 ask the student to generate?
2. Why is the correct target velocity not the arithmetic average $(1-\alpha)v_{\text{old}} + \alpha v_T$?
3. Why can A4 be trained without evaluating $\rho_{\text{old},t}$, $\rho_{T,t}$, or γ_t?
4. Which superficially similar implementations are actually A0, A1, or A3?
5. Which exact methods, tensors, index maps, and invariants must change in `src/flow_factory/trainers/xopd/trainer.py`?

1.2 Start with one conditional flow

Fix a prompt or condition $C = c$. A deterministic flow model samples an initial latent $Z \sim \rho_{\text{base}}(\cdot \mid c)$ and solves

$$\frac{\mathrm{d}X_t}{\mathrm{d}t} = v(t, X_t, c), \qquad X_0 = Z, \qquad t \in [0,1]. \tag{1}$$

We use the noise-to-data orientation: $t = 0$ is base noise and $t = 1$ is the generated sample. Schedulers that run from 1 to 0 differ only by a change of time orientation.

At outer iteration k, three fields matter:

$$v_{\text{old}} = v_{\theta_k}, \qquad v_T = \text{frozen teacher}, \qquad v_\theta = \text{trainable current student}. \tag{2}$$

The old and teacher ODEs push the same base law into two paths of time marginals, $\rho_{\text{old},t}$ and $\rho_{T,t}$.

1.3 Do not mix the three mathematical layers

The arm names used below are:

- **A0**: direct teacher-field regression on old-student states;
- **A1**: a quadratic field-space trust region or arithmetic field barycenter;
- **A3**: SDE transition-mixture P-OPD;
- **A4**: the ODE marginal-mixture CFM construction developed here.

Figure 1 locates each arm on its actual mathematical object.

For an ODE Euler step,

$$K_j(\mathrm{d}y \mid x, t) = \delta_{x + h\, v_j(t,x)}(\mathrm{d}y). \tag{3}$$

The direct KL between two unequal Dirac kernels is infinite. More specifically, if

$$Q_\alpha = (1-\alpha)\delta_{F_{\text{old}}(x)} + \alpha\delta_{F_T(x)} \tag{4}$$

and $F_{\text{old}}(x) \neq F_T(x)$, then

$$\mathrm{KL}(\delta_y \| Q_\alpha) = \begin{cases} -\log(1-\alpha), & y = F_{\text{old}}(x), \\ -\log\alpha, & y = F_T(x), \\ +\infty, & \text{otherwise}. \end{cases} \tag{5}$$

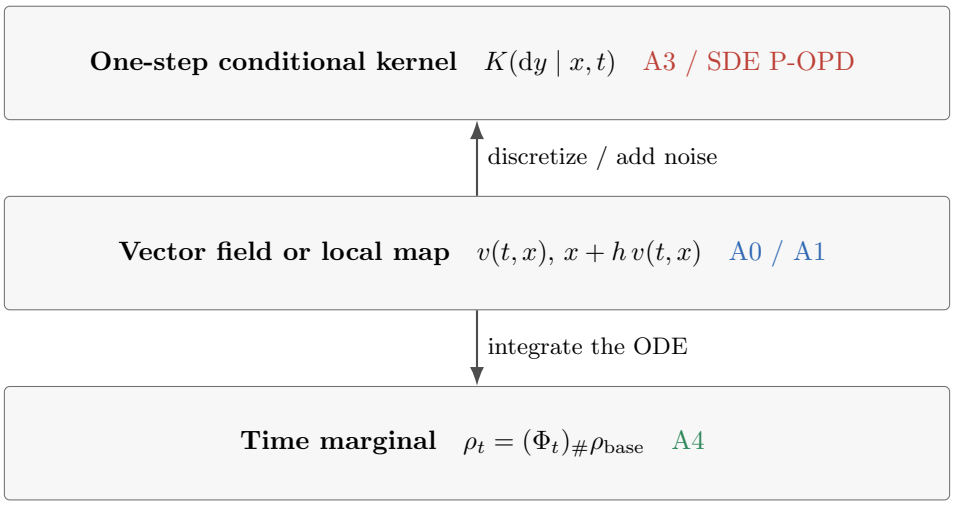

Figure 1: The three layers must not be interchanged. A3 mixes stochastic one-step kernels, A0/A1 compare deterministic field outputs, and A4 mixes the distributions of states reached at each time.

There is no smooth transition-likelihood trust region around the old map. This is the obstruction that prevents the exact SDE responsibility gate from surviving the zero-noise limit.

A4 changes the probability object. ODE *marginals* can have ordinary densities even though their conditional transitions are Dirac measures. The target is therefore a mixture of $\rho_{\text{old},t}$ and $\rho_{T,t}$, not a mixture of their deterministic next-state atoms.

One-sentence definition

A4 chooses the old or teacher ODE once per generated trajectory, uses the selected trajectory state and velocity as a CFM training pair, and fits one deterministic student field whose time marginals equal the old/teacher marginal mixture in the population limit.

1.4 Why the P-OPD result motivates A4

For shared-covariance Gaussian SDE kernels, P-OPD uses the exact transition posterior. Its detached gated quadratic matches the transition-mixture KL *first derivative locally* at the behavior student; it is not a global equality of objectives. This is the flow-transition analogue of the external probability mixture in TOP-D [3] and the transition view used by DiffusionOPD [2].

The complete-latent responsibility concentrates when the teacher and old transition means are separated. The companion analysis `docs/xopd/popd_exact_sum_gate_saturation.tex` measured a median responsibility of 8×10^{-12} and 75.4% of transitions below 0.01 at the exact latent-sum setting in a 9B→4B probe. Removing SDE noise does not repair this problem: it makes the transition supports singular.

A4 keeps ODE sampling efficiency and an exact probability-mixture target, but pays for a different resource: samples from the teacher trajectory marginal. It therefore gives up strict student-only on-policy sampling whenever $\alpha > 0$.

2 Flow Matching from scratch

2.1 From a vector field to a density path

Let $\Phi_t^v(z)$ be the solution of (1) that starts at z. The density at time t is the pushforward

$$\rho_t = (\Phi_t^v)_{\#}\rho_{\text{base}}. \tag{6}$$

Probability mass is neither created nor destroyed. Infinitesimally, this is expressed by the continuity equation

$$\partial_t \rho_t(x) + \nabla_x \cdot \big(\rho_t(x) v_t(x)\big) = 0. \tag{7}$$

The product $\rho_t v_t$ is the probability *flux*: density times local velocity.

2.2 The ordinary Flow Matching loss

Suppose a target density path q_t and one velocity field u_t satisfying

$$\partial_t q_t + \nabla \cdot (q_t u_t) = 0 \tag{8}$$

are known. Flow Matching trains

$$\mathcal{L}_{\mathrm{FM}}(\theta) = \mathbb{E}_{C,\, t\sim\omega,\, X_t\sim q_t(\cdot|C)} \left[\|v_\theta(t, X_t, C) - u_t(X_t, C)\|^2 \right]. \tag{9}$$

At zero population loss, integrating $v_\theta = u$ from the same base law generates q_t, provided the ODE is well posed.

The difficulty is that $u_t(x)$ is often an intractable conditional expectation. Conditional Flow Matching (CFM) [1] replaces it by sample-wise labels.

Proposition 2.1 (The CFM regression identity)**.** *Let (X_t, V_t) be any joint training pair such that $X_t \sim q_t$ and*

$$u_t(x) = \mathbb{E}[V_t \mid X_t = x, t]. \tag{10}$$

Then

$$\begin{aligned} \mathbb{E}\,\|v_\theta(t, X_t) - V_t\|^2 = \mathbb{E}\,\|v_\theta(t, X_t) - u_t(X_t)\|^2 \\ + \mathbb{E}\,\|V_t - u_t(X_t)\|^2 . \end{aligned} \tag{11}$$

The final term is independent of θ. CFM and FM therefore have identical population gradients and minimizers.

Proof. Write

$$v_\theta - V_t = (v_\theta - u_t) + (u_t - V_t)$$

and expand the square. Conditioned on (t, X_t), the cross term is zero because

$$\mathbb{E}[u_t(X_t) - V_t \mid t, X_t] = 0.$$

The remaining two squared terms give (11). □

2.3 Event sum, event mean, and RMS

For latent shape (C, H, W) or (N_{tokens}, C), let D be the number of non-batch elements. Two common losses are

$$\ell_{\text{sum}} = \sum_{i=1}^{D} e_i^2, \qquad \ell_{\text{mean}} = \frac{1}{D}\sum_{i=1}^{D} e_i^2, \qquad \mathrm{RMS}(e) = \sqrt{\ell_{\text{mean}}}. \tag{12}$$

Multiplying the entire objective by $1/D$ does not change its population minimizer at a fixed shape, but it changes gradient and trust-radius scales. The implementation uses the event mean for stable cross-resolution optimization and logs both RMS and L2 scales.

Listing 1: Equivalent target, different optimization scale.

```
error = velocity_student.float() - velocity_target.float()
event_axes = tuple(range(1, error.ndim))

loss_sum_per_sample = error.square().sum(dim=event_axes)   # joint event scale
loss_mean_per_sample = error.square().mean(dim=event_axes) # resolution-stable scale
error_rms = loss_mean_per_sample.sqrt()
```

Scale rule

Use one convention consistently in the loss, trust radius, diagnostics, and ablations. Do not call an event mean a joint likelihood. A4 itself is a marginal CFM construction; its practical mean-MSE normalization is an optimization choice, not a density-ratio approximation.

3 Constructing the A4 marginal path

3.1 Two source ODEs

Fix $C = c$ and suppress it from the notation. The old and teacher source paths satisfy

$$\begin{aligned} \partial_t \rho_{\text{old},t} + \nabla \cdot (\rho_{\text{old},t} v_{\text{old}}) &= 0, \\ \partial_t \rho_{T,t} + \nabla \cdot (\rho_{T,t} v_T) &= 0, \\ \rho_{\text{old},0} = \rho_{T,0} &= \rho_{\text{base}}. \end{aligned} \tag{13}$$

Choose $\alpha \in [0, 1]$. The branch variable used below is independent of (Z, t) conditional on C. Define the target marginal

$$q_t = (1 - \alpha)\rho_{\text{old},t} + \alpha \rho_{T,t}. \tag{14}$$

At every t, q_t literally samples the old marginal with probability $1 - \alpha$ and the teacher marginal with probability α.

3.2 The source-mixture flux selects a velocity

The mixture density alone is not enough; a flow also needs a flux. The source trajectories select the natural mixture flux

$$q_t u_t = (1 - \alpha)\rho_{\text{old},t} v_{\text{old}} + \alpha \rho_{T,t} v_T. \tag{15}$$

Proposition 3.1 (The mixture path is a valid Flow Matching path)**.** *The pair* (q_t, u_t) *satisfies*

$$\partial_t q_t + \nabla \cdot (q_t u_t) = 0, \qquad q_0 = \rho_{\text{base}}. \tag{16}$$

Proof. Substitute (14) and (15):

$$\begin{aligned} \partial_t q_t + \nabla \cdot (q_t u_t) = (1 - \alpha) &\left[\partial_t \rho_{\text{old},t} + \nabla \cdot (\rho_{\text{old},t} v_{\text{old}})\right] \\ &+ \alpha \left[\partial_t \rho_{T,t} + \nabla \cdot (\rho_{T,t} v_T)\right] = 0. \end{aligned}$$

The initial equality follows because both source ODEs start from the same base law. □

A density path can admit another velocity that differs from u_t by a q_t-divergence-free field. Thus u_t is not the only imaginable field for q_t; it is the specific source-mixture velocity learned by the two-source CFM joint distribution.

3.3 Marginal responsibility

Where $q_t(x) > 0$, define

$$\gamma_t(x) = \frac{\alpha \rho_{T,t}(x)}{(1-\alpha)\rho_{\text{old},t}(x) + \alpha \rho_{T,t}(x)}. \tag{17}$$

Dividing (15) by q_t gives

$$\boxed{u_t(x) = (1-\gamma_t(x))v_{\text{old}}(t,x) + \gamma_t(x) v_T(t,x)}. \tag{18}$$

The coefficient is state dependent:

- where the old marginal dominates, $\gamma_t(x) \approx 0$ and $u_t \approx v_{\text{old}}$;
- where the teacher marginal dominates, $\gamma_t(x) \approx 1$ and $u_t \approx v_T$;
- where both marginals overlap, the velocity interpolates according to posterior branch probability.

This is why the constant arithmetic average $(1-\alpha)v_{\text{old}} + \alpha v_T$ is generally wrong. Figure 2 shows the joint sampling process that selects this particular flux.

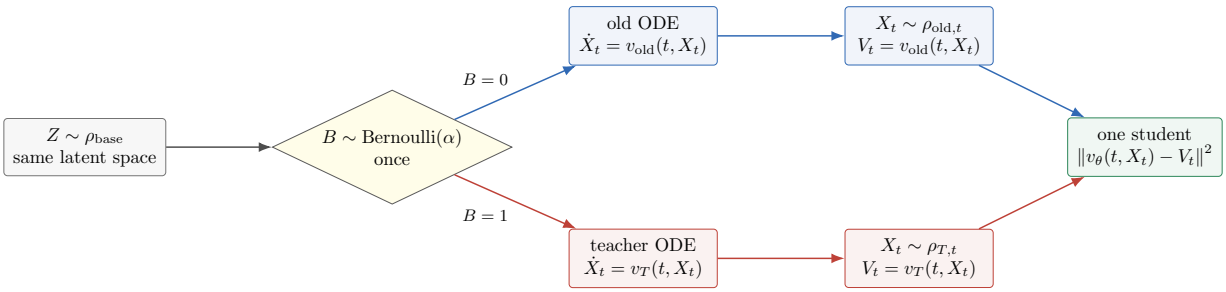

Figure 2: A4 samples the branch once for the entire trajectory. The branch determines both the state distribution and the velocity label. Only the student on the right receives gradients.

3.4 One-dimensional intuition

Figure 3 shows a schematic equal-variance example. The prior branch weight is $\alpha = 0.4$. At points near the old mode the posterior teacher responsibility is small; near the teacher mode it is large. At a point where the two source densities are equal, $\gamma_t(x) = \alpha$.

4 Why two-source CFM needs no density ratio

4.1 The training joint distribution

The direct formula (18) appears impractical because modern flow-model densities are not available. The CFM identity solves this by sampling the latent branch variable explicitly:

$$B \sim \text{Bernoulli}(\alpha), \qquad X_t = \Phi_t^B(Z), \qquad V_t = \begin{cases} v_{\text{old}}(t, X_t), & B = 0, \\ v_T(t, X_t), & B = 1. \end{cases} \tag{19}$$

Marginalizing B gives $X_t \sim q_t$. Bayes' rule gives

$$\Pr(B = 1 \mid X_t = x, t) = \frac{\alpha \rho_{T,t}(x)}{q_t(x)} = \gamma_t(x). \tag{20}$$

Therefore

$$\mathbb{E}[V_t \mid X_t = x, t] = (1-\gamma_t(x))v_{\text{old}}(t,x) + \gamma_t(x) v_T(t,x) = u_t(x). \tag{21}$$

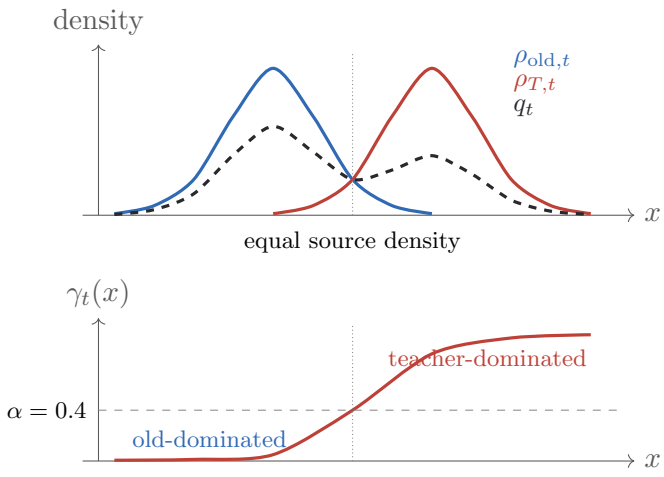

Figure 3: Schematic source densities, their marginal mixture, and the posterior teacher responsibility. The curves are illustrative rather than measured model densities.

Proposition 4.1 (The exact A4 CFM identity)**.** *Define*

$$\mathcal{L}_{\text{A4}}(\theta) = \mathbb{E}_{C,\,t,\,B,\,Z} \left\| v_\theta(t, X_t, C) - V_t \right\|^2 . \tag{22}$$

Then

$$\begin{aligned} \mathcal{L}_{\text{A4}}(\theta) &= \mathbb{E}_{C,\,t,\,X_t \sim q_t} \left\| v_\theta(t, X_t, C) - u_t(X_t, C) \right\|^2 \\ &\quad + \mathcal{C}_{\text{var}}, \end{aligned} \tag{23}$$

$$\mathcal{C}_{\text{var}} = \mathbb{E}_{C,\,t,\,X_t \sim q_t} \left[\gamma_t(X_t)(1 - \gamma_t(X_t)) \left\| v_T(t, X_t) - v_{\text{old}}(t, X_t) \right\|^2 \right] . \tag{24}$$

The variance term is independent of θ, so A4 CFM and marginal FM have identical population gradients and minimizers.

Proof. Proposition 2.1 applies because (21) identifies u_t as the conditional mean label. For a two-point random label, the conditional variance is

$$\gamma_t(1 - \gamma_t) \left\| v_T - v_{\text{old}} \right\|^2 ,$$

which gives (24). □

4.2 The central implementation insight

The density responsibility appears in the *analytic population minimizer*, but not in the Monte Carlo training recipe:

1. sample B;
2. generate a state from the corresponding ODE;
3. use that ODE's velocity as the detached label;
4. regress the current student onto the label.

Ordinary square-loss regression learns the conditional average automatically.

Listing 2: Minimal conceptual A4 training step.

```
# Conceptual code, independent of Flow-Factory plumbing.
branch = torch.rand(batch_size, device=device) < alpha
```

```
3
4  # The branch is fixed for the complete trajectory.
5  x_t, velocity_target = rollout_selected_source(
6      branch=branch,
7      initial_noise=initial_noise,
8      condition=condition,
9      training_time=training_time,
10 )
11 velocity_target = velocity_target.detach()
12
13 velocity_student = student.predict_velocity(
14     t=training_time,
15     latents=x_t,
16     condition=condition,
17 )
18 event_axes = tuple(range(1, velocity_student.ndim))
19 loss_per_sample = (
20     velocity_student.float() - velocity_target.float()
21 ).square().mean(dim=event_axes)
22 loss = loss_per_sample.mean()
```

No classifier and no fake-score network

A4 does not require a classifier for ρ_T/ρ_{old}, a CNF log-density evaluator, or a DMD-style online fake score. It requires the ability to *sample both source trajectories*. The hidden branch label supplies the posterior responsibility implicitly through conditional regression.

4.3 What the variance term means

$\mathcal{C}_{\text{var}}$ is irreducible label ambiguity. At a state where both branches have non-negligible density but prescribe different velocities, the learner observes two possible labels. The square-loss optimum is their posterior average $u_t(x)$.

This is not an optimization bug. It is exactly the velocity needed to transport the mixture marginal. It can nevertheless be difficult for a finite network when the responsibility changes sharply across state space.

5 What "exact" means, and where it stops

5.1 Exact population realization

Assume:

1. both source densities and fluxes are sufficiently smooth;
2. $q_t(x) > 0$ on the relevant region;
3. u_t is continuous in t, Lipschitz in x, and has at most linear growth;
4. the model can represent u_t and population optimization reaches it;
5. the fitted representative is regular enough to define a unique ODE.

Then

$$\dot{X}_t = u_t(X_t), \qquad X_0 \sim \rho_{\text{base}} \tag{25}$$

has marginals

$$(\Phi_t^u)_{\#}\rho_{\text{base}} = q_t = (1-\alpha)\rho_{\text{old},t} + \alpha\rho_{T,t}. \tag{26}$$

This is exactness at the *time-marginal* level. The deterministic u-trajectory law is not the same as the random path law that chooses old or teacher once; only their one-time marginals agree.

5.2 The proximal contraction

Assume $\rho_{\mathrm{old},t}$ is absolutely continuous with respect to $\rho_{T,t}$ and the displayed KL is finite. For the Wasserstein statement, let $p \geq 1$ and assume both laws have finite p-th moments. Then, at every time,

$$\|q_t - \rho_{T,t}\|_1 = (1-\alpha)\,\|\rho_{\mathrm{old},t} - \rho_{T,t}\|_1\,, \tag{27}$$

$$\mathrm{KL}(q_t\|\rho_{T,t}) \leq (1-\alpha)\mathrm{KL}(\rho_{\mathrm{old},t}\|\rho_{T,t}), \tag{28}$$

$$W_p^p(q_t,\rho_{T,t}) \leq (1-\alpha)W_p^p(\rho_{\mathrm{old},t},\rho_{T,t}). \tag{29}$$

The first identity follows by factoring the signed density difference. The KL inequality follows from convexity in its first argument. The Wasserstein bound follows by coupling an α fraction of teacher mass to itself and transporting only the remaining old mass.

If every outer iteration fits the mixture path exactly and refreshes $\rho_{\mathrm{old},t} \leftarrow \rho_{\theta,t}$, then

$$\|\rho_{k+1,t} - \rho_{T,t}\|_1 = (1-\alpha)\,\|\rho_{k,t} - \rho_{T,t}\|_1\,. \tag{30}$$

This is the clean deterministic mixture-contraction story. A real LoRA update introduces projection and optimization error.

5.3 Finite capacity

Let $\widehat{v}$ be the trained field and

$$e_t(x) = \widehat{v}(t,x) - u_t(x). \tag{31}$$

If both laws have finite second moments, the fitting error below is integrable, $\widehat{v}$ is L-Lipschitz, and the exact and approximate ODEs start from the same noise, then a Grönwall coupling gives

$$W_2^2(\widehat{\rho}_t, q_t) \leq t\int_0^t e^{2L(t-s)}\mathbb{E}_{X_s\sim q_s}\,\|e_s(X_s)\|^2\,\mathrm{d}s. \tag{32}$$

Consequently,

$$W_2(\widehat{\rho}_t,\rho_{T,t}) \leq W_2(\widehat{\rho}_t,q_t) + (1-\alpha)^{1/2}W_2(\rho_{\mathrm{old},t},\rho_{T,t}) \tag{33}$$

for $p=2$. The first term is the price of finite fitting; the second is the ideal proximal contraction.

5.4 Low overlap and stiffness

Where both source densities are positive,

$$\nabla\gamma_t = \gamma_t(1-\gamma_t)\,(\nabla\log\rho_{T,t} - \nabla\log\rho_{\mathrm{old},t})\,. \tag{34}$$

Low-overlap marginals can create a thin boundary where γ_t switches rapidly and u_t changes from v_{old} to v_T. The training labels remain valid, but a small student or a coarse ODE solver can smooth or misintegrate that boundary.

Do not transfer ideal guarantees to a finite run

Equations (27)–(29) describe the target mixture q_t. They do not prove that a finite LoRA student, finite batch, sparse timestep subsample, or imperfect optimizer achieves the same contraction. The measured CFM fitting error and rollout drift must be logged separately.

6 Methods that look like A4 but are not A4

6.1 A compact comparison

method	sampled states	target	mathematical object
A0 direct OPD	$X_t \sim \rho_{\mathrm{old},t}$	$v_T(t, X_t)$	old-occupancy field regression
A1 field barycenter	$X_t \sim \rho_{\mathrm{old},t}$	$(1-\alpha)v_{\mathrm{old}} + \alpha v_T$	quadratic field geometry
A3 P-OPD	old SDE transition (X_t, Y)	Gaussian transition responsibility	one-step stochastic-kernel mixture
A4	$X_t \sim (1-\alpha)\rho_{\mathrm{old},t} + \alpha\rho_{T,t}$	velocity of the sampled source branch	ODE time-marginal mixture

6.2 Wrong variant 1: teacher on old states

Listing 3: This is A0, not A4.

```
# WRONG if the intended method is A4.
x_t = old_trajectory.state_at(t)
velocity_target = teacher.predict_velocity(t=t, latents=x_t)
loss = mse(student.predict_velocity(t=t, latents=x_t), velocity_target)
```

The teacher target is valid, but every query state still follows $\rho_{\mathrm{old},t}$. No sample ever comes from $\rho_{T,t}$, so the training state marginal is not q_t.

Listing 4: The source branch controls both state and target.

```
branch = sample_once_for_the_trajectory(alpha)
if branch == OLD:
    x_t = old_trajectory.state_at(t)
    velocity_target = old_trajectory.velocity_at(t)
else:
    x_t = teacher_trajectory.state_at(t)
    velocity_target = teacher_trajectory.velocity_at(t)
```

6.3 Wrong variant 2: re-sample the branch every Euler step

If independent branch labels B_ℓ are drawn at every step,

$$X_{\ell+1} = X_\ell + h_\ell v_{B_\ell}(t_\ell, X_\ell), \qquad B_\ell \overset{\text{iid}}{\sim} \mathrm{Bernoulli}(\alpha), \tag{35}$$

then the conditional mean field is

$$\bar{v} = (1-\alpha)v_{\mathrm{old}} + \alpha v_T. \tag{36}$$

The switching variance vanishes with the step size, so the continuous-time limit is the arithmetic-average ODE, not the A4 marginal-mixture ODE. Their gap is

$$\bar{v} - u_t = (\alpha - \gamma_t)(v_T - v_{\mathrm{old}}). \tag{37}$$

Figure 4 contrasts the two branch lifetimes.

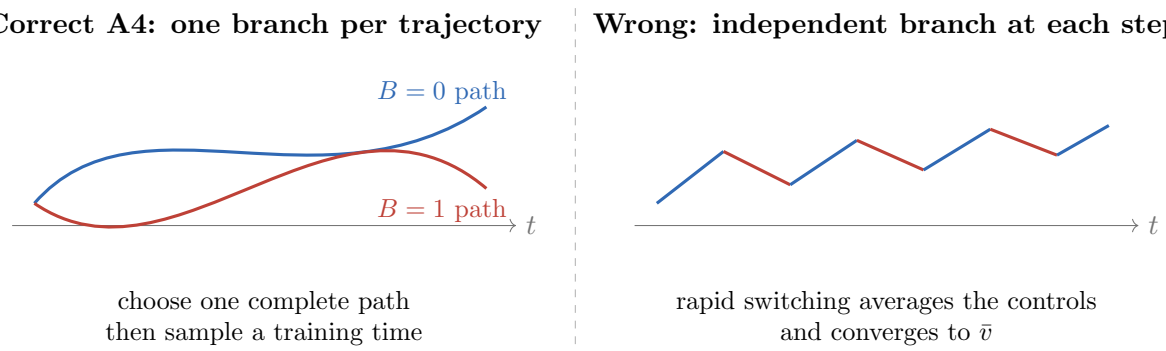

Figure 4: A path-law mixture chooses one source trajectory. Independent per-step switches solve a different problem and converge to the A1-style arithmetic field.

Listing 5: Per-step branch sampling changes the target method.

```
# WRONG: this converges to the arithmetic-average field.
for timestep_index in range(num_steps):
    branch = torch.rand(batch_size) < alpha  # re-drawn every step
    velocity = torch.where(branch, velocity_teacher, velocity_old)
    latents = latents + dt[timestep_index] * velocity
```

6.4 A3/P-OPD is a different probability mixture

A3 uses a sampled SDE next state Y and computes

$$\gamma_T(Y) = \frac{\alpha p_T(Y \mid X_t)}{(1-\alpha)p_{\text{old}}(Y \mid X_t) + \alpha p_T(Y \mid X_t)}. \tag{38}$$

This is an exact *transition* posterior under shared positive covariance. A4 instead uses the posterior branch probability of a *time marginal*, but never evaluates it. Reusing the current P-OPD responsibility helper would implement A3 again, not A4.

7 Mapping A4 to the current Flow-Factory code

7.1 Direct/P-OPD L1 data flow and the A4 split

The relevant implementation lives in:

file / symbol	current role
`src/flow_factory/trainers/xopd/trainer.py: sample`	student rollout; stores selected trajectory states and optional P-OPD callbacks
`src/flow_factory/trainers/xopd/trainer.py: _precompute_teacher_means`	teacher forward on *student rollout states*
`src/flow_factory/trainers/xopd/trainer.py: _optimize_train_pass`	student forward, transition-mean loss, backward, optimizer step
`src/flow_factory/models/flux/flux2\_klein.py: inference`	full ODE/SDE rollout with `TrajectoryCollector` and `CallbackCollector`
`src/flow_factory/models/flux/flux2\_klein.py: use_teacher_transformer`	no-grad whole-transformer teacher swap with autocast-cache protection
`src/flow_factory/utils/trajectory\_collector.py: CallbackCollector`	stacks selected per-step outputs into batch-first callback tensors

In direct and P-OPD modes, `sample()` runs the student:

Listing 6: Direct/P-OPD L1 sampling, abbreviated from `trainer.py`.

```
sample_kwargs = {
    **self.training_args,
    "compute_log_prob": False,
    "trajectory_indices": trajectory_indices,
    **batch,
}
if self._is_popd:
    sample_kwargs["extra_call_back_kwargs"] = [
        "next_latents_mean", "std_dev_t", "dt",
    ]
sample_batch = self.adapter.inference(**sample_kwargs)
```

Direct/P-OPD optimization then calls `_precompute_teacher_means`. That pre-pass evaluates the teacher at X_t^{old}, followed by a student gradient pass at the same states. It is correct for direct XOPD and transition P-OPD, but it does not sample $\rho_{T,t}$. Implemented A4 branches before this path and does not run that pre-pass. Figure 5 shows the structural difference.

7.2 The key callback already exists

Each FLUX.2-Klein denoising step computes the CFG-combined velocity and passes the same tensor to the scheduler. The tensor key is `noise_pred`; the existing callback collector can store it without another model forward:

Listing 7: Existing callback contract in `flux2_klein.py`.

```
return_kwargs = list(
    set(["next_latents", "log_prob", "noise_pred"] + extra_call_back_kwargs)
)
output = self._forward(..., return_kwargs=return_kwargs)

callback_collector.collect_step(
    step_idx=i,
    output=output,
    keys=extra_call_back_kwargs,
    capturable={"noise_level": current_noise_level},
```

```
)
```

GRPO already uses the same mechanism for v-based current–old KL:

Listing 8: Existing precedent in `trainers/grpo.py`.

```
extra_call_back_kwargs = ["noise_pred"]
...
old_noise_pred = batch["noise_pred"][
    :, callback_index_map[timestep_index]
]
sq = (output.noise_pred - old_noise_pred) ** 2
```

For A4, the callback is the exact detached scheduler-space source prediction used to advance the selected rollout. No optimization-time teacher velocity pre-pass is required.

Time-orientation sign. FLUX.2-Klein steps through decreasing scheduler noise σ:

$$x_{\text{next}} = x + \texttt{noise\_pred}\,\Delta\sigma, \qquad \Delta\sigma < 0. \tag{39}$$

The mathematical convention in this note uses noise-to-data time $t = 1 - \sigma$, so

$$v_t = -\,\texttt{noise\_pred}. \tag{40}$$

The implementation may compare `noise_pred` directly because the common sign cancels:

$$\|(-n_\theta) - (-n_B)\|^2 = \|n_\theta - n_B\|^2 . \tag{41}$$

The callback is therefore loss-equivalent to the mathematical velocity, but not identical in sign under the displayed time orientation.

7.3 Two different compact index maps

Selective trajectory storage creates two maps:

map	indexes	use at denoising step `ti`
`latent_index_map`	state positions, length $T+1$	`all_latents[:, latent_index_map[ti]]`
`callback_index_map`	transition steps; (B, T) after stacking	normalize rows, validate, then index `noise_pred`

Each sample stores a one-dimensional callback map inside its extra dictionary. Because the map is not a shared field, sample stacking produces shape (B, T). Normalize it only after asserting that all rows agree:

Listing 9: Normalize the stacked callback map safely.

```
callback_index_map = batch["callback_index_map"]
if callback_index_map.ndim != 2:
    raise ValueError(
        "expected stacked callback_index_map with shape (B, T), "
        f"got shape={tuple(callback_index_map.shape)}."
    )
expected = callback_index_map[:1].expand_as(callback_index_map)
if not torch.equal(callback_index_map, expected):
    raise ValueError(
        "callback_index_map must be identical across the micro-batch, "
        f"got shape={tuple(callback_index_map.shape)}."
    )
callback_index_map = callback_index_map[0]  # (T,)
```

An uncollected step maps to -1 and must raise rather than silently selecting the final compact entry.

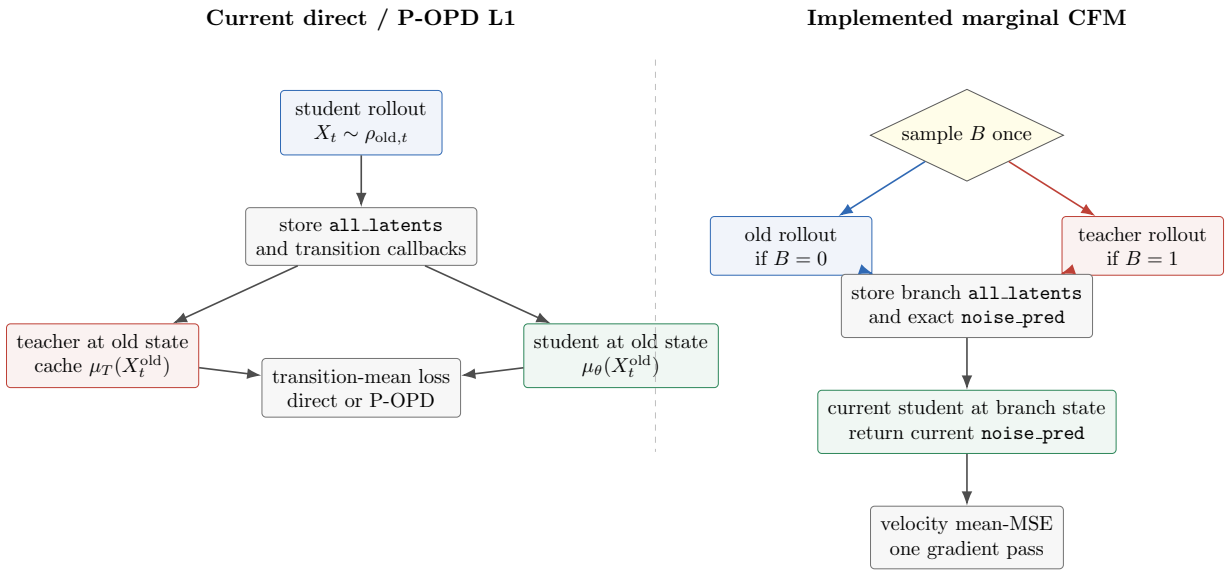

Figure 5: Direct/P-OPD XOPD queries the teacher on old states. Implemented A4 routes each sample through one source trajectory, captures that source's velocity, and removes the teacher pre-pass.

7.4 One subtle conditioning issue

Teacher rollout must use its dedicated prompt embeddings and text ids. The returned sample, however, stores whichever embeddings were active during that inference call. Before teacher-branch samples enter the student gradient pass, their ordinary student-conditioning fields must be restored from the original dataloader rows:

$$\text{teacher trajectory state/target} \quad + \quad \text{student prompt conditioning for } v_\theta. \tag{42}$$

Failing to restore them would train the smaller student with teacher-encoder embeddings and would no longer match normal student inference.

8 Implemented same-VAE XOPD path

Status

A4 is implemented as `xopd_target_mode: marginal_cfm`. The production path is in `src/flow_factory/trainers/xopd/trainer.py` and `src/flow_factory/trainers/xopd/common.py`. Focused coverage is in `tests/test_marginal_cfm_trainer.py`, `tests/test_xopd_helpers.py`, and `tests/test_xopd_configs.py`.

8.1 Configuration

Use a dedicated alpha rather than overloading `popd_alpha`; the latter is a transition mixture prior with different diagnostics and validity conditions.

Listing 10: Implemented same-VAE A4 configuration.

```
train:
  trainer_type: xopd
  xopd_target_mode: marginal_cfm
  marginal_cfm_alpha: 0.5
```

```yaml
  # A4 always matches source velocity.
  xopd_dk_space: v
  normalize_d_k: false
  pathwise_coef: 1.0

  # Existing timestep selection and one-step-per-epoch geometry remain.
  num_xopd_steps: 4
  xopd_step_sampling: stratified

scheduler:
  dynamics_type: ODE
  noise_level: 0.0
```

The implementation enforces these fail-fast checks:

1. `dynamics_type == "ODE"`;
2. $0 \leq$ `marginal_cfm_alpha` ≤ 1 and finite;
3. same VAE / same latent state space; reject cross-VAE transport and pixel loss;
4. `xopd_dk_space == "v"` and `normalize_d_k == false`;
5. teacher and student rollout grids, latent shapes, and selected steps agree;
6. every selected callback index is nonnegative and every target is finite.

Implemented touch points. The mode is wired through the complete rollout/replay path:

1. the target-mode literal and dedicated A4 alpha live in XOPD training arguments;
2. the trainer derives `_is_marginal_cfm` at initialization;
3. target-mode-specific validation preserves direct and P-OPD behavior;
4. A4 validation enforces the ODE, same-VAE, and v-space matrix;
5. both `sample()` and `optimize()` dispatch through A4-specific target handling;
6. canonical validation and matched smoke configs serialize the target fields;
7. the direct-XOPD validation-D_k evaluator is skipped in A4 mode, because it queries the teacher on student states rather than evaluating held-out marginal CFM.

8.2 Step 1: draw one branch before rollout

Listing 11: Implemented deterministic branch draw.

```python
branches = draw_marginal_cfm_branches(
    batch_size,
    alpha=self.training_args.marginal_cfm_alpha,
    seed=self.training_args.seed,
    epoch=self.epoch,
    batch_index=batch_index,
)
```

Draw the labels before calling `inference`. Split the dataloader batch into old and teacher row subsets. Each row runs exactly one ODE trajectory:

$$\text{expected rollout cost per sample} = (1-\alpha)C_{\text{old}} + \alpha C_T. \tag{43}$$

Running a student trajectory for every row and an additional teacher trajectory for $B = 1$ rows would cost $C_{\text{old}} + \alpha C_T$ and store unnecessary paths.

Distributed call-order invariant

For the first FSDP implementation, all ranks must draw the *same branch pattern* and have the same local batch geometry. The seed above intentionally excludes rank. Then every rank enters the old and teacher transformer collectives in the same order with the same subset sizes. Adding rank to the seed can make one rank skip a teacher call while another enters it, which can deadlock. The cost is correlated branch labels across ranks; independent routing would require an always-call/padded collective design.

8.3 Step 2: route the two subsets and capture the source velocity

`_slice_marginal_cfm_batch_rows` preserves row-aligned tensor/list fields and shared objects, while `_sample_marginal_cfm_batch` handles empty subsets, restores original row order before metadata stitching, and offloads the merged samples through the normal sample lifecycle. Nested I2I inputs and positive/negative prompt embeddings plus text ids are covered by the same routing and restoration path.

Listing 12: Implemented A4 sampling structure, abbreviated.

```python
def _sample_marginal_cfm_batch(
    self,
    batch: Dict[str, Any],
    batch_index: int,
    trajectory_indices: List[int],
) -> List[BaseSample]:
    batch_size = int(batch["prompt_embeds"].shape[0])
    branches = draw_marginal_cfm_branches(
        batch_size,
        alpha=self.training_args.marginal_cfm_alpha,
        seed=self.training_args.seed,
        epoch=self.epoch,
        batch_index=batch_index,
    )
    result: List[Optional[BaseSample]] = [None] * batch_size

    for use_teacher in (False, True):
        row_indices = torch.nonzero(
            branches == use_teacher, as_tuple=False
        ).flatten()
        if row_indices.numel() == 0:
            continue

        subset = self._slice_marginal_cfm_batch_rows(
            batch, row_indices, batch_size
        )
        infer_kwargs = {
            **self.training_args,
            **subset,
            "compute_log_prob": False,
            "trajectory_indices": trajectory_indices,
```

```
            "extra_call_back_kwargs": ["noise_pred"],
            "guidance_scale": (
                self.teacher_gs if use_teacher else self.student_gs
            ),
        }

        if use_teacher:
            self._route_marginal_cfm_teacher_conditioning(
                infer_kwargs, subset
            )
            infer_kwargs = filter_kwargs(
                self.adapter.inference, **infer_kwargs
            )
            with self.adapter.use_teacher_transformer():
                subset_samples = self.adapter.inference(**infer_kwargs)
            self._restore_marginal_cfm_student_conditioning(
                subset_samples, subset
            )
        else:
            infer_kwargs = filter_kwargs(
                self.adapter.inference, **infer_kwargs
            )
            subset_samples = self.adapter.inference(**infer_kwargs)

        expected_count = int(row_indices.numel())
        if len(subset_samples) != expected_count:
            raise RuntimeError(
                "marginal CFM subset rollout count mismatch: "
                f"use_teacher={use_teacher}, "
                f"expected={expected_count}, "
                f"received={len(subset_samples)}."
            )
        for original_row, sample in zip(
            row_indices.tolist(), subset_samples
        ):
            sample.extra_kwargs["marginal_cfm_branch"] = torch.tensor(
                use_teacher, dtype=torch.bool
            )
            result[original_row] = sample

    if any(sample is None for sample in result):
        missing = [i for i, sample in enumerate(result) if sample is None]
        raise RuntimeError(
            "marginal CFM rollout did not produce one sample per input row; "
            f"missing_rows={missing}, batch_size={batch_size}."
        )

    merged: List[BaseSample] = []
    for sample in result:
        if sample is None:
            raise RuntimeError(
                "marginal CFM internal merge invariant failed after "
                "the missing-row check."
            )
        merged.append(sample)
    stitch_batch_metadata(batch, merged)
    self._maybe_offload_samples_to_cpu(merged)
    return merged
```

Teacher conditioning helpers. The routing helper must fail if teacher-side positive conditioning is absent, and must also require teacher-side negative conditioning whenever teacher CFG is active:

Listing 13: Implemented teacher-conditioning replacement rules, abbreviated.

```python
def _route_marginal_cfm_teacher_conditioning(
    self,
    inference_kwargs: Dict[str, Any],
    subset: Dict[str, Any],
) -> None:
    required = ("teacher_prompt_embeds", "teacher_text_ids")
    missing = [key for key in required if subset.get(key) is None]
    if missing:
        raise ValueError(
            "marginal CFM teacher rollout requires teacher conditioning, "
            f"missing_keys={missing!r}."
        )

    device = self.accelerator.device
    inference_kwargs["prompt_embeds"] = subset[
        "teacher_prompt_embeds"
    ].to(device)
    inference_kwargs["text_ids"] = subset["teacher_text_ids"].to(device)
    inference_kwargs.pop("prompt_ids", None)

    negative_keys = (
        "teacher_negative_prompt_embeds",
        "teacher_negative_text_ids",
    )
    if self.teacher_gs > 1.0:
        missing_negative = [
            key for key in negative_keys if subset.get(key) is None
        ]
        if missing_negative:
            raise ValueError(
                "marginal CFM teacher CFG requires negative teacher "
                f"conditioning, missing_keys={missing_negative!r}, "
                f"teacher_guidance_scale={self.teacher_gs}."
            )
        inference_kwargs["negative_prompt_embeds"] = subset[
            "teacher_negative_prompt_embeds"
        ].to(device)
        inference_kwargs["negative_text_ids"] = subset[
            "teacher_negative_text_ids"
        ].to(device)
        inference_kwargs.pop("negative_prompt_ids", None)
    else:
        for key in (
            "negative_prompt_embeds",
            "negative_text_ids",
            "negative_prompt_ids",
        ):
            inference_kwargs.pop(key, None)
```

After inference, restore these student-side sample fields from the subset rows:

`prompt_ids`, `prompt_embeds`, `text_ids`,
`negative_prompt_ids`, `negative_prompt_embeds`, `negative_text_ids`.

Keep the teacher-specific fields untouched for diagnostics. Row slicing/restoration must also preserve nested I2I conditions rather than assuming every value is a dense tensor.

Three details are correctness-critical:

1. `extra_call_back_kwargs=["noise_pred"]` is necessary. Including `noise_pred` in internal `return_kwargs` alone does not make `CallbackCollector` store it.
2. Teacher samples keep teacher trajectory latents and callback targets, but their ordinary prompt fields are restored to the student conditioning.
3. The teacher context is no-grad and uses `adapter.use_teacher_transformer()`, which already swaps away from the wrapped student and disables the autocast cache for the scope.

8.4 Step 3: align the state and callback target

After `BaseSample.stack`, expected shapes are

field	shape	meaning
`all_latents`	$(B, T_x, \dots)$	selected state positions
`latent_index_map`	$(T+1,)$	full position $\rightarrow$ compact state index
`noise_pred`	$(B, T_v, \dots)$	selected source velocities
`callback_index_map`	(B, T) before normalization	full step $\rightarrow$ compact callback index
`marginal_cfm_branch`	$(B,)$	diagnostic source label

Listing 14: Correct state/velocity alignment after map normalization.

```python
latent_compact_index = int(latent_index_map[timestep_index])
callback_compact_index = int(callback_index_map[timestep_index])

if latent_compact_index < 0 or callback_compact_index < 0:
    raise RuntimeError(
        "marginal CFM selected an uncollected trajectory step: "
        f"timestep_index={timestep_index}, "
        f"latent_compact_index={latent_compact_index}, "
        f"callback_compact_index={callback_compact_index}."
    )

latents = batch["all_latents"][:, latent_compact_index]
velocity_target = batch["noise_pred"][
    :, callback_compact_index
].detach()
```

Do not index `noise_pred` with `latent_index_map`: callbacks index transitions, whereas latent storage indexes positions.

8.5 Step 4: keep the existing optimizer shell

The safest implementation does *not* create a second copy of the optimizer loop. Extend the existing `_optimize_train_pass` pathwise-loss dispatch, while preserving its exact ordering:

$$\begin{aligned}&\text{accumulate} \rightarrow \text{student forward} \rightarrow \text{pathwise} + \text{auxiliary} + \text{KL losses}\\ &\rightarrow \text{backward} \rightarrow \text{clip / step / zero.}\end{aligned}$$

The student return-key helper must always request `noise_pred` in A4 mode.

Listing 15: Implemented A4 branch inside the existing optimizer shell, abbreviated.

```
with self.accelerator.accumulate(*self.adapter.trainable_components):
    # Build batched t, t_next, latents, and next_latents exactly as
    # the current _optimize_train_pass does.
    forward_kwargs = self._build_forward_kwargs(
        batch=batch,
        t=t,
        t_next=t_next,
        latents=latents,
        next_latents=next_latents,
        compute_log_prob=False,
        return_kwargs=self._student_return_kwargs_for_train(),
        guidance_scale=self.student_gs,
    )
    student_out = self.adapter.forward(**forward_kwargs)

    if self._is_marginal_cfm:
        callback_index = resolve_marginal_cfm_callback_index(
            callback_index_map,
            timestep_index=timestep_index,
            callback_count=int(batch["noise_pred"].shape[1]),
        )
        target_noise_pred = batch["noise_pred"][
            :, callback_index
        ].detach()
        if student_out.noise_pred is None:
            raise RuntimeError(
                "marginal CFM requires current student `noise_pred`; "
                f"got None at timestep_index={timestep_index}."
            )
        d_k_grad = compute_marginal_cfm_velocity_loss(
            student_out.noise_pred,
            target_noise_pred,
        )
        diagnostics = compute_marginal_cfm_diagnostics(
            student_noise_pred=student_out.noise_pred,
            target_noise_pred=target_noise_pred,
            teacher_branch_mask=batch["marginal_cfm_branch"],
        )
        self._append_marginal_cfm_diagnostics(
            loss_info,
            diagnostics,
            batch["marginal_cfm_branch"],
        )
    elif self._is_popd:
        # Existing P-OPD branch remains unchanged.
        d_k_grad = compute_popd_pathwise_term(...)
    else:
        # Existing direct / pixel branches remain unchanged.
        d_k_grad = compute_direct_pathwise_term(...)

    pathwise_loss = d_k_grad.mean()
    loss = self.pathwise_coef * pathwise_loss

    # The existing inline MoE/MoF auxiliary-loss blocks add to loss here.
    if self.enable_kl_loss:
        kl_div, kl_loss = self._compute_kl_anchor(
            student_out, forward_kwargs
```

```
        )
        loss = loss + kl_loss

    # Backward occurs only after every differentiable term is added.
    self.accelerator.backward(loss)
    if self.accelerator.sync_gradients:
        grad_norm = self._clip_grad_norm_ep_aware(...)
        self.optimizer.step()
        self.optimizer.zero_grad()
        # Keep the existing reduce/log/step bookkeeping.
```

The P-OPD/direct branches and inline auxiliary blocks are abbreviated in the listing; the A4 helper names are the production symbols. The number of backward calls remains

$$\texttt{num\_batches\_per\_epoch} \times \texttt{num\_train\_timesteps},$$

so the existing one-optimizer-step-per-epoch GAS invariant remains valid.

8.6 Dispatch changes

The high-level branch in `optimize()` skips only target precomputation; every mode then enters the shared optimizer shell:

Listing 16: Implemented target-mode dispatch, abbreviated.

```
if self._is_marginal_cfm:
    mu_teacher_list = None
    popd_cache_list = None
    callback_index_map = normalize_callback_index_map(
        batch["callback_index_map"]
    )
else:
    mu_teacher_list = self._precompute_teacher_means(...)
    popd_cache_list = (
        self._precompute_popd_step_caches(...)
        if self._is_popd else None
    )
    callback_index_map = None

loss_info = self._optimize_train_pass(
    ...,
    mu_teacher_list=mu_teacher_list,
    popd_cache_list=popd_cache_list,
    callback_index_map=callback_index_map,
)
```

A4 must not call `_precompute_teacher_means`: the exact branch target has already been captured during the source rollout. The shared optimizer signature must make `mu_teacher_list` and `popd_cache_list` optional, and every access to them must remain inside the direct/P-OPD branches. The student return-key helper must add `noise_pred` whenever `self._is_marginal_cfm` is true.

8.7 Compute and memory

For B samples and K trained steps:

design	rollout forwards	optimize forwards	trajectory storage
direct XOPD	B student trajectories	BK teacher + BK student	states
implemented A4	$(1-\alpha)B$ old + αB teacher trajectories	BK student	states + T_v velocities
dual-rollout A4	B old + αB teacher trajectories	BK student	extra teacher states

The implemented design removes the teacher pre-pass but stores one velocity tensor for every callback step. Do not assume $T_v = K$: the current trajectory-index expansion can collect neighboring positions and make T_v approach $2K$. Either introduce a separate callback-step selector or measure and report the actual compact callback length. With all steps stored, callback storage is roughly another trajectory-sized tensor.

9 Validation, diagnostics, and implementation checklist

9.1 Unit tests

The focused implementation tests cover:

1. **Branch frequency**: a large deterministic draw approaches α and is repeatable for the same seed/epoch/batch index.
2. **Boundary cases**: $\alpha = 0$ produces only old branches; $\alpha = 1$ produces only teacher branches.
3. **Branch lifetime**: one label is attached to the entire sample and never redrawn per timestep.
4. **Index alignment**: real `Flux2KleinSample` objects passed through `BaseSample.stack` produce a (B, T) callback map, normalize correctly, retrieve the intended state/target, and raise on -1.
5. **Gradient ownership**: target callbacks and teacher parameters have no gradient; current student parameters do.
6. **Source correctness**: teacher branch states come from teacher inference, not teacher forwards on old states.
7. **Conditioning restoration**: positive and negative prompt embeddings plus text ids are restored to student values after teacher rollout.
8. **Storage lifecycle**: branch labels, callback tensors, and maps survive CPU offload, device reload, shuffling, and stacking.
9. **Optimizer integration**: MoE/MoF auxiliaries and the optional KL anchor contribute gradients before backward; GAS still yields one optimizer step per epoch.
10. **Distributed call-order protocol**: the rank-independent branch draw and fixed old-then-teacher subset order are deterministic, including empty subsets.
11. **Shape/finite guards**: mismatched shape, NaN, or Inf raises an informative error.

Listing 17: Representative implemented helper-level assertions.

```
def test_marginal_cfm_loss_only_gradients_student():
    velocity_student = torch.tensor([[1.0, 2.0]], requires_grad=True)
    velocity_target = torch.tensor([[0.0, 4.0]], requires_grad=False)
```

```
    loss = (velocity_student - velocity_target).square().mean()
    loss.backward()

    assert velocity_student.grad is not None
    assert velocity_target.grad is None


def test_callback_map_shape_after_real_sample_stack():
    map_per_sample = torch.tensor([-1, -1, 0, -1])
    samples = [
        Flux2KleinSample(
            extra_kwargs={
                "callback_index_map": map_per_sample,
                "marginal_cfm_branch": torch.tensor(False),
            },
        ),
        Flux2KleinSample(
            extra_kwargs={
                "callback_index_map": map_per_sample.clone(),
                "marginal_cfm_branch": torch.tensor(True),
            },
        ),
    ]

    batch = BaseSample.stack(samples)
    assert batch["callback_index_map"].shape == (2, 4)
    callback_map = normalize_callback_index_map(
        batch["callback_index_map"]
    )
    assert torch.equal(callback_map, map_per_sample)
    assert batch["marginal_cfm_branch"].shape == (2,)
```

9.2 GPU smoke tests

The checked-in pair holds the prompt source, model, timestep schedule, batch/GAS geometry, and optimizer settings fixed. Only the target behavior changes:

Listing 18: Matched eight-GPU smoke commands.

```
ff-train xopd_configs/ode_pathwise/_TEST_9b_4b_marginal_cfm_smoke.yaml
ff-train xopd_configs/ode_pathwise/_TEST_9b_4b_direct_ctrl_marginal_cfm_smoke.yaml
```

The A4 config uses $\alpha = 0.5$; $\alpha = 0$ and $\alpha = 1$ routing boundaries are covered by CPU unit tests. These commands are documented but were **NOT RUN locally** and are **not evidence of passing distributed validation**. The real smoke requires an eight-GPU host. Check:

- exactly one optimizer step per epoch;
- no call to `_precompute_teacher_means` in A4 mode;
- finite loss and gradient norm;
- callback/state maps align at every selected step;
- teacher swap scope restores the wrapped student component;
- MoE/MoF and KL-anchor losses are included before backward;
- the legacy direct validation-D_k path is disabled or explicitly relabeled in A4 mode;

- eval generation still uses the ordinary student ODE.

9.3 Diagnostics

The shared `reduce_loss_info` reducer preserves scalar keys and expands every per-sample vector into `_min`, `_max`, `_mean`, and `_std` series. The overall objective and matching term therefore remain the scalar `train/loss` and `train/d_k` keys; there is no `train/marginal_cfm/loss`. In the table, A4 base names are under `train/marginal_cfm/`; "four stats" means all four suffixes above.

metric	purpose
`train/loss`, `train/d_k`	scalar overall objective and CFM matching term
`train/loss/\{ti\}`, `train/d_k/\{ti\}`	scalar per-timestep objective and matching term
`callback_count`	measure actual compact callback storage
`teacher_branch_fraction` (four stats)	verify Bernoulli routing and distributed aggregation
`loss_old`, `loss_teacher` (four stats)	branch-conditioned fitting difficulty; absent for an empty branch
`target_velocity_rms` (four stats)	resolution-stable target scale
`target_velocity_l2` (four stats)	full event scale
`student_target_gap_rms` (four stats)	direct CFM fitting error
`train/grad_norm`	distinguish small CFM error from optimizer clipping

Do *not* log a transition `gamma`: A4 never computes one. A density-responsibility estimate would be an optional research diagnostic, not part of the training algorithm.

9.4 Implementation checklist

Minimum correct same-VAE A4

- ✓ ODE-only, shared latent space, shared timestep grid.
- ✓ Draw one branch label per full trajectory before rollout.
- ✓ Run only the selected old or teacher trajectory for each row.
- ✓ Capture the selected source's CFG-combined `noise_pred`.
- ✓ Restore student conditioning fields on teacher-trajectory samples.
- ✓ Align states with `latent_index_map` and targets with `callback_index_map`.
- ✓ Detach target velocities; backpropagate only through the current student.
- ✓ Skip the existing teacher-on-old-state pre-pass.
- ✓ Preserve accumulation, auxiliary losses, KL anchor, and optimizer boundaries.
- ✓ Preserve rank-synchronous old/teacher collective call order.
- ✓ Skip direct-XOPD validation-D_k in A4 mode.
- ✓ Log branch fraction, per-branch losses, callback count, RMS/L2 scales, and gradient norm.

9.5 Final mental model

$$\boxed{\begin{array}{c}\text{sample source branch} \implies \text{sample source trajectory} \implies \text{regress source velocity} \\ \text{CFM conditional averaging} \implies u_t(x) \implies q_t = (1-\alpha)\rho_{\text{old},t} + \alpha\rho_{T,t}.\end{array}} \tag{44}$$

The key is not to calculate the marginal responsibility. The key is to sample the joint distribution whose conditional mean *is* the responsible mixture velocity.

References

[1] Yaron Lipman, Ricky T. Q. Chen, Heli Ben-Hamu, Maximilian Nickel, and Matt Le. Flow Matching for Generative Modeling. *International Conference on Learning Representations*, 2023. `https://arxiv.org/abs/2210.02747`.

[2] Quanhao Li, Junqiu Yu, Kaixun Jiang, et al. DiffusionOPD: A Unified Perspective of On-Policy Distillation in Diffusion Models. 2026. `https://arxiv.org/abs/2605.15055`.

[3] Zhengpeng Xie, Li Lyna Zhang, Zeke Xie, and Mao Yang. Trust Region Policy Distillation. 2026. `https://arxiv.org/abs/2607.04751`.